

目录

- 1 实验目的**
- 2 实验平台及环境**
- 3 实验内容**
- 4 实验步骤与结果分析**
 - 4.1 前置步骤 —— 数据库连接
 - 4.2 程序设计与实现
 - 4.2.1 总体架构设计
 - 4.2.2 JDBC API 使用映射
 - 4.2.3 数据库访问流程实现
 - 4.2.4 4.4 程序运行演示
 - 4.2.5 用户交互式程序
 - 4.2.6 程序关键特性
- 5 实验总结与心得体会**
- 6 附录A: 交互式数据库应用程序源代码**
- 7 附录B: 演示程序源代码**

实验七 数据库接口实验

1 实验目的

1. 华为的 GaussDB 支持基于 C、Java 等应用程序的开发。了解它相关的系统结构和相关概念，有助于更好地开发和使用 GaussDB 数据库。
2. 通过实验了解通用数据库应用编程接口 ODBC/JDBC 的基本原理和实现机制，熟悉连接 ODBC/JDBC 接口的语法和使用方法。
3. 熟练 GaussDB 的各种连接方式与常用工具的使用。
4. 利用 C 语言(或其它支持 ODBC/JDBC 接口的高级程序设计语言)编程实现简单的数据库应用程序，掌握基于 ODBC 的数据库访问基本原理和方法。

2 实验平台及环境

本实验环境为华为云 GaussDB 数据库，具体的配置如表 2-1 所示：

表 2-1 实验环境

配置项	参数
华为云	软件版本：GaussDB v2.0-8.210.0 设备型号：GaussDB 8 核 164 G
数据库	PostgreSQL
本机操作系统	macOS Sequoia 15.7.1

3 实验内容

1. 本实验内容通过使用 ODBC/JDBC 等驱动开发应用程序。
2. 连接语句访问数据库接口，实现对数据库中的数据进行操作（包括增、删、改、查等）。
3. 要求能够通过编写程序访问到华为数据库，该实验重点在于 ODBC/JDBC 数据源配置和高级语言（C/C++/JAVA/PYTHON）的使用。

4 实验步骤与结果分析

为保证报告叙述的连续性，我将“实验步骤”与“实验结果分析”合并为一节，在每个操作步骤之后给出对应的结果和对实验结果的分析。

4.1 前置步骤 —— 数据库连接

 **Tip**

这一节花了较大的篇幅介绍我是如何在 macOS 系统下实现连接的，评阅时可快进或跳过，我在实验建议中对这一节的作用也给出了说明。

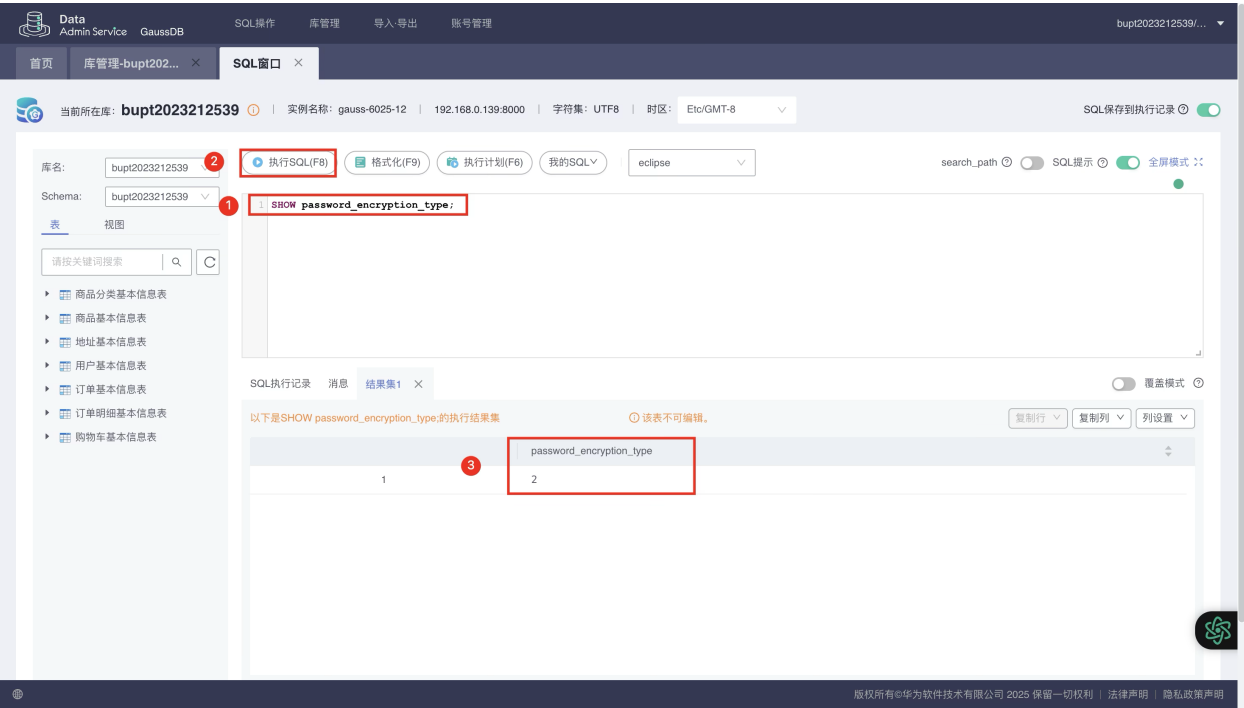
实验指导书中提供的步骤是针对 Windows 操作系统，macOS 系统可以通过安装虚拟机等方式解决，但我进行了一定的折腾，并给出一种通过 Python/Java 应用程序连接的可行方法（C 语言程序暂时无法实现）见下文。

法一：安装兼容 macOS 系统的开源驱动程序

写在开头，这种方法需要让云实例的管理员（老师）手动在云端修改数据库的加密策略为兼容模式。通过以下 SQL 语句可以查看当前实例的加密策略。

1

SHOW password_encryption_type;



The screenshot shows the GaussDB SQL interface. The SQL statement 'SHOW password_encryption_type;' is entered in the SQL window. The execution result is displayed in a table with one row and one column, showing the value '2'. Red boxes and numbers highlight the SQL statement, the 'Execute SQL' button, and the result value.

password_encryption_type
2

图 4-1 数据库实例加密策略

执行结果如图 4-1 所示，对于我们的实验实例 `gauss-6025-12`，加密策略为 2，表示仅支持 SHA256，由于华为没有发布 macOS 版本的官方 ODBC 驱动（只有 Windows 和 Linux 版），通过法一安装开源驱动进行连接的方式不适用，请改为使用下文的法二，或者联系老师，将加密策略改为 1，即同时存储 SHA256 和 MD5，对应的 SQL 语句如下：

```
1 ALTER SYSTEM SET password_encryption_type = 1;
```

假如上面的条件满足，可以执行以下步骤：

1. 通过 Homebrew 安装开源驱动管理器 **unixODBC** 和数据库驱动 **psqlodbc**：

```
1 # 1. 安装 unixODBC (ODBC 驱动管理器)
2 brew install unixodbc
3
4 # 2. 安装 PostgreSQL 的 ODBC 驱动 (psqlodbc)
5 brew install psqlodbc
```

2. 安装完毕后，输入 `odbcinst -j`，查看配置文件位置，如图 4-2 所示；

```
> odbcinst -j
unixODBC 2.3.14
DRIVERS.....: /opt/homebrew/etc/odbcinst.ini
SYSTEM DATA SOURCES: /opt/homebrew/etc/odbc.ini
FILE DATA SOURCES..: /opt/homebrew/etc/ODBCDataSources
USER DATA SOURCES..: /Users/yokumi/.odbc.ini
SQLULEN Size.....: 8
SQLLEN Size.....: 8
SQLSETPOSIROW Size.: 8
```

图 4-2 数据库驱动管理器配置文件位置

3. 输入 `brew list psqlodbc` 查看数据库驱动程序位置，如图 4-3 所示；

```
> brew list psqlodbc
/opt/homebrew/Cellar/psqlodbc/17.00.0006/lib/ (2 files)
/opt/homebrew/Cellar/psqlodbc/17.00.0006/sbom.spdx.json
```

图 4-3 数据库驱动程序位置

3. 注册驱动，打开 `odbcinst.ini`，添加以下内容后保存（请根据本机实际路径进行修改），如图 4-4 所示；

```
[PostgreSQL Unicode]
Description = PostgreSQL ODBC driver (Unicode version)
Driver      = /opt/homebrew/Cellar/psqlodbc/17.00.0006/lib/psqlodbcw.so
Setup       = /opt/homebrew/Cellar/psqlodbc/17.00.0006/lib/psqlodbcw.so
FileUsage   = 1
```

图 4-4 注册驱动

4. **配置数据源**：这里和 Windows 下“配置 ODBC 数据源”等价。打开 `odbc.ini`，添加以下内容后保存（请根据实际情况进行修改），如图 4-5 所示；

```
[gaussdb_mac]
Driver       = PostgreSQL Unicode
Servername   = 113.45.130.136
Port         = 8000
Database     = bupt2023212539
Username     = bupt2023212539
Password     = 
SSLmode      = prefer
ReadOnly     = 0
```

图 4-5 配置用户数据源

其中：

- `Servername` 为实例 IP，我们的 6025 实例为 113.45.130.136；
 - `Port` 为实例端口，我们的 6025 实例为 8000；
 - `Database` 为学生数据库名；
 - `Username` 为登录该数据库的用户名；
 - `Password` 为登录该数据库的用户的密码；
 - `SSLmode` 设置为 prefer，表示根据实际情况选择；
5. **验证连接**：输入 `isql -v gaussdb_mac` 验证连接，如图 4-6 所示，这里因为远程实例的加密策略限制，无法通过开源驱动进行连接，所以报错“**none of the server's SASL authentication mechanisms are supported**”；

```
> isql -v gaussdb_mac
[08001][unixODBC]connection to server at "113.45.130.136", port 8000 failed: none of the server's SASL authentication mechanisms are supported
[ISQL]ERROR: Could not SQLConnect
```

图 4-6 验证连接

6. （如果上一步可以连接）通过以下命令，安装 Python 依赖：

```
1 pip install pyodbc
```

7. 编写以下 Python 程序，并运行测试连接：

```
1 import pyodbc
2
3 try:
4     # 注意：DSN 必须与 odbc.ini 中配置的中括号名称一致，例如 'gaussdb_mac'
5     # 如果在 odbc.ini 里已经配好了账号密码，这里只需要 DSN
6     conn_str = 'DSN=gaussdb_mac;UID=bupt2023212539;PWD=XXXXXX'
7     conn = pyodbc.connect(conn_str)
8     cursor = conn.cursor()
```

```

9
10     cursor.execute("SELECT version();")
11     row = cursor.fetchone()
12     print("数据库版本信息:", row)
13
14     cursor.close()
15     conn.close()
16     print("数据库连接成功! ")
17
18 except Exception as e:
19     print("连接失败:", e)

```

这里，如果不调整数据库加密策略，同样会报如图 4-6 所示的错误。

法二：通过 Java 驱动连接

华为提供的 JDBC 驱动 (`gaussdbjdbc.jar`) 是跨平台的，且支持所有华为专有的加密协议。因此，macOS 上，Java 程序可以直接使用 JDBC 驱动连接，而 Python 可以通过 `JayDeBeApi` 库来调用这个 Jar 包。

通过 Java 程序连接的步骤与实验指导书类似，此处略。

通过 Python 程序连接需要通过 `JayDeBeApi` 库来调用 Jar 包：

```

1 pip install JayDeBeApi JPype1

```

编写 Python 代码如下：

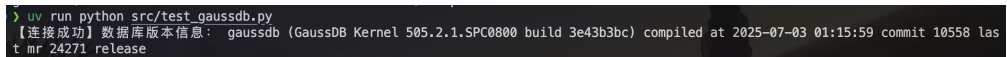
```

1 import jaydebeapi
2 import jpype
3
4 # === 配置信息 ===
5 jar_path = "驱动地址/gaussdbjdbc.jar"
6 driver_class = "com.huawei.gaussdb.jdbc.Driver"
7 jdbc_url = "jdbc:gaussdb://113.45.130.136:8000/bupt2023212539"
8 user = "bupt2023212539"
9 password = "xxxxxxx"
10
11 try:
12     # === 启动 JVM ===
13     if not jpype.isJVMStarted():
14         jpype.startJVM(
15             jpype.getDefaultJVMPath(),
16             "-ea",
17             f"-Djava.class.path={jar_path}",
18             "-Djava.util.logging.config.file=logging.properties"
19         )
20
21     # === 建立连接 ===
22     conn = jaydebeapi.connect(

```

```
23         driver_class,
24         jdbc_url,
25         [user, password],
26         jar_path
27     )
28
29     curs = conn.cursor()
30     curs.execute("SELECT version();")
31     result = curs.fetchone()
32
33     print("【连接成功】数据库版本信息：")
34     print(result[0])
35
36     curs.close()
37     conn.close()
38
39 except Exception as e:
40     print(f"连接失败：{e}")
41
42 finally:
43     if jpye.isJVMStarted():
44         jpye.shutdownJVM()
```

如果操作正确，可以观察到如图 4-7 所示，输出正确信息：



```
> uv run python src/test_gaussdb.py
【连接成功】数据库版本信息： gaussdb (GaussDB Kernel 505.2.1.SPC0800 build 3e43b3bc) compiled at 2025-07-03 01:15:59 commit 10558 las
t mr 24271 release
```

图 4-7 连接成功

4.2 程序设计与实现

4.2.1 总体架构设计

根据实验指导书的要求，本实验需要针对**一张表**进行完整的 CRUD（增删改查）操作。经过分析，我选择了实验二创建的**"商品基本信息表"**作为操作对象，原因如下：

- 1. 字段丰富（包含 product_id、name、description、price、stock、category_id 及时间戳字段）；
- 2. 业务逻辑清晰（商品管理是电商系统的核心功能）；
- 3. 操作场景直观（增加商品、修改价格/库存、删除商品、查询列表）；
- 4. 具有外键关联（category_id 与商品分类表关联，可验证数据完整性）。

程序采用**模块化、面向对象**的设计思路，整体架构如图 4-8 所示：

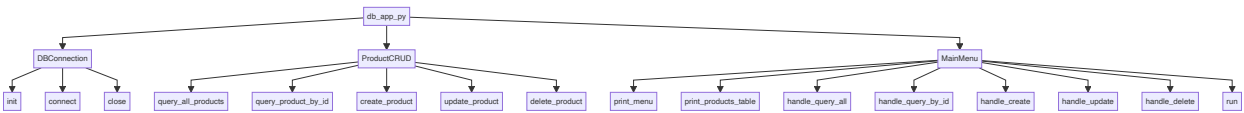


图 4-8 程序架构设计

4.2.2 JDBC API 使用映射

由于本实验采用 Python + JDBC 方案，我将 ODBC API 与 Python JDBC 操作进行了对应映射，如表 4-1 所示：

表 4-1 ODBC API 与 Python JDBC 映射关系

ODBC API (C 语言)	Python jaydebeapi 对应操作	功能说明
SQLAllocEnv	jpyype.startJVM()	初始化 JDBC 环境（启动 JVM）
SQLAllocConnect	jaydebeapi.connect()	分配连接句柄
SQLConnect / SQLDriverConnect	jaydebeapi.connect(driver, url, [user, pwd], jar)	建立数据库连接
SQLAllocStmt	conn.cursor()	创建语句句柄（游标）
SQLExecDirect	cursor.execute(sql)	执行 SQL 语句
SQLFetch / SQLFetchAdvances	cursor.fetchone() / cursor.fetchall()	获取查询结果
SQLGetData	row[column_index]	从结果集提取数据
SQLFreeStmt	cursor.close()	释放游标资源
SQLDisconnect	conn.close()	关闭数据库连接
SQLFreeConnect / SQLFreeEnv	jpyype.shutdownJVM()	释放连接和环境资源

4.2.3 数据库访问流程实现

根据实验指导书要求，数据库访问流程分为 4 个步骤，具体实现如下：

Step 1: JDBC 环境初始化

在 `DBConnection.connect()` 方法中，首先启动 JVM 并加载 GaussDB JDBC 驱动：

```
1  # Step 1: 启动 JVM
2  if not jpyype.isJVMStarted():
3      print("【Step 1】初始化 JDBC 环境...")
4      jpyype.startJVM(
5          jpyype.getDefaultJVMPath(),
6          "-ea",
7          f"-Djava.class.path={self.jar_path}",
8          "-Djava.util.logging.config.file=logging.properties"
9      )
10     self.jvm_started = True
11     print("✓ JVM 启动成功")
```

Step 2: 建立数据库连接

使用 `jaydebeapi.connect()` 建立与 GaussDB 数据库的连接，并关闭自动提交模式以支持手动事务管理：

```

1  # Step 2: 建立连接
2  print("【Step 2】建立数据库连接...")
3  self.conn = jaydebeapi.connect(
4      self.driver_class, # com.huawei.gaussdb.jdbc.Driver
5      self.jdbc_url,     # jdbc:gaussdb://113.45.130.136:8000/bupt2023212539
6      [self.user, self.password],
7      self.jar_path
8  )
9  # 关闭自动提交模式，支持手动事务管理
10 self.conn.jconn.setAutoCommit(False)
11 print("✓ 数据库连接成功")

```

Step 3: 执行 CRUD 操作

`ProductCRUD` 类实现了完整的增删改查功能。以插入操作为例，代码如下：

```

1  def create_product(self, name: str, description: str, price: float,
2      stock: int, category_id: Optional[int] = None) -> bool:
3      """
4      插入新商品
5      """
6      cursor = None
7      try:
8          cursor = self._create_cursor()
9
10         # 使用参数化查询防止 SQL 注入
11         sql = '''INSERT INTO "商品基本信息表"
12             ("name", "description", "price", "stock", "category_id")
13             VALUES (?, ?, ?, ?, ?)'''
14
15         cursor.execute(sql, [name, description, price, stock, category_id])
16         self.conn.commit() # 手动提交事务
17
18         print(f"✓ 插入成功! 商品名称: {name}")
19         return True
20
21     except Exception as e:
22         print(f"✗ 插入失败: {e}")
23         self.conn.rollback() # 回滚事务
24         return False
25     finally:
26         if cursor:
27             cursor.close()

```

其他操作（查询、更新、删除）遵循相同的模式：创建游标 → 执行 SQL → 提交/回滚 → 释放游标。

Step 4: 关闭连接并释放资源

在 `DBConnection.close()` 方法中，依次关闭数据库连接和 JVM：

```

1  def close(self):
2      """
3      关闭连接并释放资源
4      """
5      try:
6          if self.conn:
7              self.conn.close()
8              print("✓ 数据库连接已关闭")
9
10         if self.jvm_started and jpyype.isJVMStarted():
11             jpyype.shutdownJVM()
12             print("✓ JVM 已关闭")
13     except Exception as e:
14         print(f"⚠ 关闭连接时出现警告: {e}")

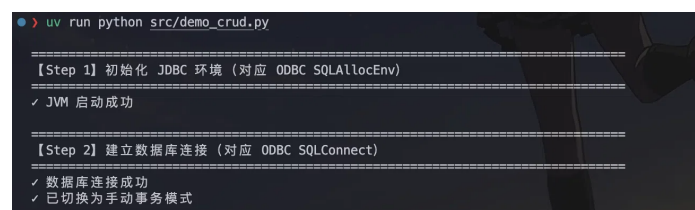
```

4.2.4 4.4 程序运行演示

为了验证程序功能，我编写了 `demo_crud.py` 自动演示脚本，按照实验指导书的逻辑顺序执行操作：

1. 查询所有商品（打印初始状态）
2. 插入测试商品
3. 再次查询（验证插入成功）
4. 修改商品信息（价格和库存）
5. 再次查询（验证修改成功）
6. 删除测试商品
7. 最终查询（验证删除成功）

运行 `uv run python src/demo_crud.py`，程序输出如图 4-9 所示：



```

> uv run python src/demo_crud.py

=====
【Step 1】初始化 JDBC 环境（对应 ODBC SQLAllocEnv）
=====
✓ JVM 启动成功

=====
【Step 2】建立数据库连接（对应 ODBC SQLConnect）
=====
✓ 数据库连接成功
✓ 已切换为手动事务模式

```

图 4-9 Step 1-2: 初始化环境与建立连接

如图 4-9 所示，程序成功初始化 JVM，加载 GaussDB JDBC 驱动，并建立数据库连接。执行第一次查询，如图 4-10 所示，显示当前数据库中共有 39 条商品记录。

【Step 3】执行数据库 CRUD 操作						
【操作 1】查询所有商品（初始状态）						
ID	商品名称	描述	价格	库存	分类ID	
1	红烧牛肉面	精选牛肉，搭配手工拉面，汤底浓郁	97.20	96	9	10
2	老北京炸酱面	传统炸酱配方，搭配黄瓜丝和豆芽	72.00	148	9	10
3	传统手工桃酥	酥脆香甜，传统工艺制作，独立包装	24.00	300	11	
4	纯棉短袖T恤	100%纯棉材质，舒适透气，适合夏季	69.00	200	12	12
5	修身直筒牛仔裤	高弹力面料，修身设计，时尚百搭	229.00	120	13	13
6	轻便透气运动鞋	网面设计，轻便舒适，适合跑步和日常穿着	349.00	80	14	14
7	时尚棒球帽	可调节帽围，遮阳防晒，多种颜色可选	59.00	150	15	15
8	苹果iPad Pro	12.9英寸全面屏，M1芯片，高性能平板电脑	7999.00	50	17	
9	苹果iPhone 14 Pro	6.1英寸超视网膜XDR显示屏，A16仿生芯片	8999.00	30	16	
10	高性能轻薄笔记本电脑	16英寸，Intel i7处理器，16GB内存	8999.00	25	18	
11	无线降噪蓝牙耳机	主动降噪，长续航，支持无线充电	699.00	60	19	
12	4K超高清智能电视	55英寸，支持HDR10，内置智能系统	5999.00	20	20	
13	双开门大容量冰箱	节能静音，风冷无霜，大容量设计	6999.00	15	21	
14	智能电饭煲	4L容量，多功能烹饪，支持预约功能	499.00	40	22	
15	快速加热微波炉	20L容量，一键加热，简单易用	699.00	30	23	
16	现代简约布艺沙发	北欧风格，可拆洗面料，舒适坐感	3999.00	10	24	
17	舒适双人床	优质松木框架，高密度海绵床垫	2999.00	15	25	
18	实木餐椅	优质橡木材质，稳固耐用，适合家庭使用	299.00	50	26	
19	简约办公桌	钢结构，简约设计，适合家庭和办公室	799.00	30	27	
20	记忆棉护颈枕头	符合人体工学设计，助眠护颈	129.00	100	28	
21	陶瓷茶杯套装	4只装，手工陶瓷，优雅设计	99.00	200	29	
22	陶瓷碗套装	6只装，手工陶瓷，简约设计	129.00	200	30	
23	陶瓷盘子套装	6只装，耐用易清洗，适合家庭使用	159.00	150	31	
24	竹制环保筷子	天然竹材，环保健康，10双装	19.00	500	32	
25	不锈钢勺子套装	4只装，304不锈钢，耐用易清洗	29.00	400	33	
26	高效去污洗衣液	3kg装，深层去污，温和不伤手	49.00	100	34	
27	滋养修护洗发水	500mL，含植物精华，修复受损发质	69.00	80	35	
28	软毛牙刷套装	4只装，超细软毛，保护牙龈	19.00	300	36	
29	美白防蛀牙膏	120g，含氟配方，有效美白防蛀	19.00	200	37	
30	纯棉毛巾套装	3条装，柔软吸水，适合家庭使用	59.00	150	38	
31	3in1速溶咖啡 100%纯正咖啡豆	三合一速溶咖啡，采用100%纯正咖啡豆研磨，口感醇厚	40.00	300	40	
32	天然草本茶包 - 90%菊花成分	富含90%菊花成分的天然草本茶包，清热去火	48.00	250	41	
33	地板清洁液 - 50%天然植物配方	含有50%天然植物配方的地板清洁液，清洁同时呵护地板	45.00	180	43	
34	新商品_咖啡豆	100%阿拉比卡咖啡豆，中度烘焙	143.84	50	1	
35	马克杯	捷克进口	200.00	600	18	
36	水晶杯	捷克进口	150.00	180	9	
37	青瓷碗	浙江龙泉	20.00	120	35	
70	合法测试商品	该商品价格大于0	10.99	100	5	
72	测试商品_233627	这是一个测试商品	999.99	100	1	

共 39 条记录

图 4-10 插入前

【操作 2】插入测试商品						
✓ 插入成功！商品名称：测试商品_154858，价格：999.99，库存：100						
【操作 3】再次查询（验证插入成功）						
ID	商品名称	描述	价格	库存	分类ID	
1	红烧牛肉面	精选牛肉，搭配手工拉面，汤底浓郁	97.20	96	9	10
2	老北京炸酱面	传统炸酱配方，搭配黄瓜丝和豆芽	72.00	148	9	10
3	传统手工桃酥	酥脆香甜，传统工艺制作，独立包装	24.00	300	11	
4	纯棉短袖T恤	100%纯棉材质，舒适透气，适合夏季	69.00	200	12	12
5	修身直筒牛仔裤	高弹力面料，修身设计，时尚百搭	229.00	120	13	13
6	轻便透气运动鞋	网面设计，轻便舒适，适合跑步和日常穿着	349.00	80	14	14
7	时尚棒球帽	可调节帽围，遮阳防晒，多种颜色可选	59.00	150	15	15
8	苹果iPad Pro	12.9英寸全面屏，M1芯片，高性能平板电脑	7999.00	50	17	
9	苹果iPhone 14 Pro	6.1英寸超视网膜XDR显示屏，A16仿生芯片	8999.00	30	16	
10	高性能轻薄笔记本电脑	16英寸，Intel i7处理器，16GB内存	8999.00	25	18	
11	无线降噪蓝牙耳机	主动降噪，长续航，支持无线充电	699.00	60	19	
12	4K超高清智能电视	55英寸，支持HDR10，内置智能系统	5999.00	20	20	
13	双开门大容量冰箱	节能静音，风冷无霜，大容量设计	6999.00	15	21	
14	智能电饭煲	4L容量，多功能烹饪，支持预约功能	499.00	40	22	
15	快速加热微波炉	20L容量，一键加热，简单易用	699.00	30	23	
16	现代简约布艺沙发	北欧风格，可拆洗面料，舒适坐感	3999.00	10	24	
17	舒适双人床	优质松木框架，高密度海绵床垫	2999.00	15	25	
18	实木餐椅	优质橡木材质，稳固耐用，适合家庭使用	299.00	50	26	
19	简约办公桌	钢结构，简约设计，适合家庭和办公室	799.00	30	27	
20	记忆棉护颈枕头	符合人体工学设计，助眠护颈	129.00	100	28	
21	陶瓷茶杯套装	4只装，手工陶瓷，优雅设计	99.00	200	29	
22	陶瓷碗套装	6只装，手工陶瓷，简约设计	129.00	200	30	
23	陶瓷盘子套装	6只装，耐用易清洗，适合家庭使用	159.00	150	31	
24	竹制环保筷子	天然竹材，环保健康，10双装	19.00	500	32	
25	不锈钢勺子套装	4只装，304不锈钢，耐用易清洗	29.00	400	33	
26	高效去污洗衣液	3kg装，深层去污，温和不伤手	49.00	100	34	
27	滋养修护洗发水	500mL，含植物精华，修复受损发质	69.00	80	35	
28	软毛牙刷套装	4只装，超细软毛，保护牙龈	19.00	300	36	
29	美白防蛀牙膏	120g，含氟配方，有效美白防蛀	19.00	200	37	
30	纯棉毛巾套装	3条装，柔软吸水，适合家庭使用	59.00	150	38	
31	3in1速溶咖啡 100%纯正咖啡豆	三合一速溶咖啡，采用100%纯正咖啡豆研磨，口感醇厚	40.00	300	40	
32	天然草本茶包 - 90%菊花成分	富含90%菊花成分的天然草本茶包，清热去火	48.00	250	41	
33	地板清洁液 - 50%天然植物配方	含有50%天然植物配方的地板清洁液，清洁同时呵护地板	45.00	180	43	
34	新商品_咖啡豆	100%阿拉比卡咖啡豆，中度烘焙	143.84	50	1	
35	马克杯	捷克进口	200.00	600	18	
36	水晶杯	捷克进口	150.00	180	9	
37	青瓷碗	浙江龙泉	20.00	120	35	
70	合法测试商品	该商品价格大于0	10.99	100	5	
72	测试商品_233627	这是一个测试商品	999.99	100	1	
77	测试商品_154858	这是一个测试商品	999.99	100	1	

共 40 条记录

图 4-11 插入后

如图 4-11 所示，程序成功插入测试商品（名称：测试商品_154858，价格：999.99，库存：100），插入后查询显示记录数增加至 40 条。

[操作 4] 修改商品 ID=77 (价格改为 1999.99, 库存改为 50)

✓ 更新成功! 商品 ID: 77, 新价格: 1999.99, 新库存: 50

[操作 5] 再次查询 (验证修改成功)

ID	商品名称	描述	价格	库存	分类ID			
1	红烧牛肉面	精选牛肉, 搭配手工拉面, 汤底浓郁	97.20	96	10			
2	老北京炸酱面	传统炸酱配方, 搭配黄瓜丝和豆芽	72.00	148	10			
3	传统手工桃酥	酥脆香甜, 传统工艺制作, 独立包装	24.00	300	11			
4	纯棉短袖T恤	100%纯棉材质, 舒适透气, 适合夏季	69.00	200	12			
5	修身直筒牛仔褲	高弹力面料, 修身设计, 时尚百搭	229.00	120	13			
6	轻便透气运动鞋	网面设计, 轻便舒适, 适合跑步和日常穿着	349.00	80	14			
7	时尚棒球帽	可调节帽围, 遮阳防晒, 多种颜色可选	59.00	150	15			
8	苹果 iPad Pro	12.9英寸全面屏, M1芯片, 高性能平板电脑	7999.00	50	17			
9	苹果 iPhone 14 Pro	6.1英寸超视网膜XDR显示屏, A16仿生芯片	8999.00	30	16			
10	高性能轻薄笔记本电脑	16英寸, Intel i7处理器, 16GB内存	8999.00	25	18			
11	无线降噪蓝牙耳机	主动降噪, 长续航, 支持无线充电	699.00	60	19			
12	4K超高清智能电视	55英寸, 支持HDR10, 内置智能系统	5999.00	20	20			
13	双开门大容量冰箱	节能静音, 风冷无霜, 大容量设计	6999.00	15	21			
14	智能电饭煲	4L容量, 多功能烹饪, 支持预约功能	499.00	40	22			
15	快速加热微波炉	20L容量, 一键加热, 简单易用	699.00	30	23			
16	现代简约布艺沙发	北欧风格, 可拆洗面料, 舒适坐感	3999.00	10	24			
17	舒适双人床	优质松木框架, 高密度海绵床垫	2999.00	15	25			
18	实木餐椅	优质橡木材质, 稳固耐用, 适合家庭使用	299.00	50	26			
19	简约办公桌	钢结构, 简约设计, 适合家庭和办公室	799.00	30	27			
20	记忆棉护颈枕头	符合人体工学设计, 助眠护颈	129.00	100	28			
21	陶瓷茶杯套装	4只装, 手工陶瓷, 优雅设计	99.00	200	29			
22	陶瓷碗套装	6只装, 手工陶瓷, 简约设计	129.00	200	30			
23	陶瓷盘子套装	6只装, 耐用易清洗, 适合家庭使用	159.00	150	31			
24	竹制环保筷子	天然竹材, 环保健康, 10双装	19.00	500	32			
25	不锈钢勺子套装	4只装, 304不锈钢, 耐用易清洗	29.00	400	33			
26	高效去污洗衣液	3kg装, 深层去污, 温和不伤手	49.00	100	34			
27	滋养修护洗发水	500ml, 含植物精华, 修复受损发质	69.00	80	35			
28	软毛牙刷套装	4只装, 超细软毛, 保护牙龈	19.00	300	36			
29	美白防蛀牙膏	120g, 含氟配方, 有效美白防蛀	19.00	200	37			
30	纯棉毛巾套装	3条装, 柔软吸水, 适合家庭使用	59.00	150	38			
31	3in1速溶咖啡 100%纯正咖啡豆	三合一速溶咖啡, 采用100%纯正咖啡豆研磨, 口感醇厚	40.00	300	40			
32	天然草本茶包 - 90%菊花成分	富含90%菊花成分的天然草本茶包, 清热去火	48.00	250	41			
33	地板清洁剂 - 50%天然植物配方	含有50%天然植物配方的地板清洁剂, 清洁同时呵护地板	45.00	180	43			
34	新商品-咖啡豆	100%阿拉比卡咖啡豆, 中度烘焙	143.84	50	1			
35	马克杯	捷克进口	200.00	600	18			
36	水晶杯	捷克进口	150.00	180	9			
37	青瓷碗	浙江龙泉	20.00	120	35			
70	合法测试商品	该商品价格大于0	10.99	100	5			
72	测试商品_233627	这是一个测试商品	999.99	100	1			
77	测试商品_154858	这是一个测试商品	1999.99	50	1			

共 40 条记录

图 4-12 更新操作与验证

如图 4-12 所示，程序成功修改 ID=77 的商品，将价格从 999.99 更新为 1999.99，库存从 100 更新为 50。再次查询验证修改生效。

[操作 6] 删除商品 ID=77

✓ 删除成功! 商品 ID: 77

[操作 7] 最终查询 (验证删除成功)

ID	商品名称	描述	价格	库存	分类ID			
1	红烧牛肉面	精选牛肉, 搭配手工拉面, 汤底浓郁	97.20	96	10			
2	老北京炸酱面	传统炸酱配方, 搭配黄瓜丝和豆芽	72.00	148	10			
3	传统手工桃酥	酥能香甜, 传统工艺制作, 独立包装	24.00	300	11			
4	纯棉短袖T恤	100%纯棉材质, 舒适透气, 适合夏季	69.00	200	12			
5	修身直筒牛仔裤	高弹力面料, 修身设计, 时尚百搭	229.00	120	13			
6	轻便透气运动鞋	网面设计, 轻便舒适, 适合跑步和日常穿着		349.00	80	14		
7	时尚棒球帽	可调节帽围, 遮阳防晒, 多种颜色可选	59.00	150	15			
8	苹果 iPad Pro	12.9英寸全面屏, M1芯片, 高性能平板电脑	7999.00	50	17			
9	苹果 iPhone 14 Pro	6.1英寸超视网膜XDR显示屏, A16仿生芯片	8999.00	30	16			
10	高性能轻薄笔记本电脑	16英寸, Intel i7处理器, 16GB内存	8999.00	25	18			
11	无线降噪蓝牙耳机	主动降噪, 长续航, 支持无线充电	699.00	60	19			
12	4K超高清智能电视	55英寸, 支持HDR10, 内置智能系统	5999.00	20	20			
13	双开门大容量冰箱	节能静音, 风冷无霜, 大容量设计	6999.00	15	21			
14	智能电饭煲	4L容量, 多功能烹饪, 支持预约功能	499.00	40	22			
15	快速加热微波炉	20L容量, 一键加热, 简单易用	699.00	30	23			
16	现代简约布艺沙发	北欧风格, 可拆洗面料, 舒适坐感	3999.00	10	24			
17	舒适双人床	优质松木框架, 高密度海绵床垫	2999.00	15	25			
18	实木餐椅	优质橡木材质, 稳固耐用, 适合家庭使用	299.00	50	26			
19	简约办公桌	钢木结构, 简约设计, 适合家庭和办公室	799.00	30	27			
20	记忆棉护颈枕头	符合人体工学设计, 助眠护颈	129.00	100	28			
21	陶瓷茶杯套装	4只装, 手工陶瓷, 优雅设计	99.00	200	29			
22	陶瓷碗套装	6只装, 手工陶瓷, 简约设计	129.00	200	30			
23	陶瓷盘子套装	6只装, 耐用易清洗, 适合家庭使用	159.00	150	31			
24	竹制环保筷子	天然竹材, 环保健康, 10双装	19.00	500	32			
25	不锈钢勺子套装	4只装, 304不锈钢, 耐用易清洗	29.00	400	33			
26	高效去污洗衣液	3kg装, 深层去污, 温和不伤手	49.00	100	34			
27	滋养修护洗发水	500ml, 含植物精华, 修复受损发质	69.00	80	35			
28	软毛牙刷套装	4只装, 超细软毛, 保护牙龈	19.00	300	36			
29	美白防蛀牙膏	120g, 含氟配方, 有效美白防蛀	19.00	200	37			
30	纯棉毛巾套装	3条装, 柔软吸水, 适合家庭使用	59.00	150	38			
31	3in1速溶咖啡100%纯正咖啡豆	二合一速溶咖啡, 采用100%纯正咖啡豆研磨, 口感醇厚	40.00	300	40			
32	天然草本茶包 - 90%菊花成分	富含90%菊花成分的天然草本茶包, 清热去火	48.00	250	41			
33	地板清洁剂 - 50%天然植物配方	含有50%天然植物配方的地板清洁剂, 清洁同时呵护地板	45.00	180	43			
34	新商品 - 咖啡豆	100%阿拉比卡咖啡豆, 中度烘焙	143.84	50	1			
35	马克杯	捷克进口	200.00	600	18			
36	水晶杯	捷克进口	150.00	180	9			
37	青瓷碗	浙江龙泉	20.00	120	35			
70	合法测试商品	该商品价格大于0	10.99	100	5			
72	测试商品_233627	这是一个测试商品	999.99	100	1			

共 39 条记录

图 4-13 删除操作与验证

如图 4-13 所示，程序成功删除 ID=77 的测试商品，最终查询显示记录数恢复为 39 条。随后程序执行 Step 4，依次关闭数据库连接和 JVM，释放所有资源。整个 CRUD 流程完整、正确，如图 4-14 所示。

```
=====
【Step 4】释放资源 (对应 ODBC SQLDisconnect/SQLFreeEnv)
=====
✓ 数据库连接已关闭
✓ JVM 已关闭
=====
```

图 4-14 资源释放

4.2.5 用户交互式程序

我还实现了交互式菜单程序 `product_manager.py`，用户可以通过菜单选择操作，如图 4-15 所示：



图 4-15 交互式菜单界面

用户输入对应编号即可执行查询、插入、更新、删除等操作，程序界面友好，错误提示清晰。由于篇幅限制，我这里不再展示其他操作。

4.2.6 程序关键特性

- 1. **异常处理完善**：所有数据库操作均包含 `try-except-finally` 块，确保异常情况下游标和连接资源正常释放；
- 2. **事务管理**：关闭自动提交模式，支持手动 `commit()` / `rollback()`，保证数据一致性；
- 3. **参数化查询**：使用占位符 `?` 传递参数，防止 SQL 注入攻击；
- 4. **类型兼容性处理**：JDBC 返回的 Java 对象需转换为 Python 原生类型，避免格式化错误；
- 5. **资源释放保证**：`finally` 块确保游标、连接、JVM 按序释放，避免资源泄漏。

5 实验总结与心得体会

通过本次实验，我深入学习了数据库应用程序接口（JDBC/ODBC）的工作原理和使用方法，掌握了基于高级编程语言访问数据库的完整流程。

通过本次实验，我对 JDBC/ODBC 的工作原理有了更深刻的理解，JDBC / ODBC 采用驱动管理器 + 驱动程序的分层架构，应用程序通过标准 API 与驱动管理器交互，驱动管理器再调用具体的数据库驱动程序。这种设计实现了应用程序与数据库的解耦，使得同一套代码可以通过更换驱动连接不同的数据库，体现了"接口与实现分离"的软件工程思想。本实验中，我使用华为 GaussDB 的 JDBC 驱动（`gaussdbjdbc.jar`），通过 Java 虚拟机（JVM）加载驱动类，建立与远程数据库的 TCP 连接，最终通过游标（Cursor）执行 SQL 语句并获取结果集。这一过程让我对数据库连接池、预编译语句、事务管理等概念有了更直观的认识。

实验中也遇到了不少问题，包括 Java 对象与 Python 格式化字符串的兼容性问题，解决方案是在格式化前，先将 Java 对象转换为 Python 原生类型；以及自动提交模式导致无法手动管理事务，这是因为 JDBC 连接默认开启自动提交模式，每条 SQL 语句执行后自动提交，无法手动控制事务边界，因此需要在建立连接后立即关闭自动提交模式，此后可以手动调用 `commit()` 和 `rollback()` 管理事务。

综上，本次实验让我从理论到实践全面掌握了 JDBC/ODBC 接口的使用，学会了编写健壮、安全的数据库应用程序。在实验过程中，我遇到了平台兼容性、类型转换、事务管理等多个问题，通过查阅官方文档和反复调试，最终一一攻克。

一些对实验的建议：

- 提供跨平台连接指导：**实验指导书仅针对 Windows 平台，建议补充 macOS、Linux 环境下的连接方案，降低学生的试错成本。我在前文中提到的方法经个人验证完全可行，可以补充到之后的实验指导书中，也可以鼓励学弟学妹自由进行探索。
- 增加异常场景测试：**实验可要求测试一些异常场景，例如违反外键约束（删除被订单引用的商品），违反唯一性约束（插入重复的 email）或并发修改冲突（两个连接同时更新同一商品），以帮助我们更好地理解数据库约束和事务机制。

6 附录A：交互式数据库应用程序源代码

```
1 import jaydebeapi
2 import jpyype
3 import os
4 from datetime import datetime
5 from typing import Optional, List, Tuple
6 from dotenv import load_dotenv
7
8
9 class DBConnection:
10     """数据库连接管理模块"""
11
12     def __init__(self, jar_path: str, jdbc_url: str, user: str, password:
13         str):
14         """
15         初始化数据库连接配置
16         """
17         self.jar_path = jar_path
18         self.driver_class = "com.huawei.gaussdb.jdbc.Driver"
19         self.jdbc_url = jdbc_url
20         self.user = user
21         self.password = password
22         self.conn = None
23         self.jvm_started = False
24
25     def connect(self):
26         """
27         建立数据库连接
28         """
29         try:
30             # Step 1: 启动 JVM
31             if not jpyype.isJVMStarted():
32                 print("【Step 1】初始化 JDBC 环境...")
33                 jpyype.startJVM(
34                     jpyype.getDefaultJVMPath(),
35                     "-ea",
36                     f"-Djava.class.path={self.jar_path}",
37                     "-Djava.util.logging.config.file=logging.properties"
38                 )
39                 self.jvm_started = True
40                 print("✓ JVM 启动成功")
41
42             # Step 2: 建立连接
43             print("【Step 2】建立数据库连接...")
44             self.conn = jaydebeapi.connect(
45                 self.driver_class,
46                 self.jdbc_url,
47                 [self.user, self.password],
48                 self.jar_path
49             )
50             # 关闭自动提交模式，支持手动事务管理
51             self.conn.jconn.setAutoCommit(False)
```

```

51         print("✓ 数据库连接成功")
52         return self.conn
53
54     except Exception as e:
55         print(f"✗ 连接失败: {e}")
56         raise
57
58     def close(self):
59         """
60         关闭连接并释放资源
61         """
62         try:
63             if self.conn:
64                 self.conn.close()
65                 print("✓ 数据库连接已关闭")
66
67             if self.jvm_started and jpye.isJVMStarted():
68                 jpye.shutdownJVM()
69                 print("✓ JVM 已关闭")
70
71         except Exception as e:
72             print(f"⚠ 关闭连接时出现警告: {e}")
73
74
75     class ProductCRUD:
76         """商品表 CRUD 操作模块"""
77
78         def __init__(self, connection):
79             """初始化 CRUD 操作类"""
80             self.conn = connection
81
82         def _create_cursor(self):
83             """
84             创建游标
85             """
86             return self.conn.cursor()
87
88         def query_all_products(self) -> List[Tuple]:
89             """
90             查询所有商品
91             """
92             cursor = None
93             try:
94                 cursor = self._create_cursor()
95
96                 # 执行查询
97                 sql = 'SELECT "product_id", "name", "description", "price",
98 "stock", "category_id", "create_time", "update_time" FROM "商品基本信息表"
99 ORDER BY "product_id"'
100                 cursor.execute(sql)
101
102                 # 获取所有结果
103                 results = cursor.fetchall()
104                 return results

```

```

104         except Exception as e:
105             print(f"❌ 查询失败: {e}")
106             return []
107         finally:
108             if cursor:
109                 cursor.close()
110
111     def query_product_by_id(self, product_id: int) -> Optional[Tuple]:
112         """根据 ID 查询单个商品"""
113         cursor = None
114         try:
115             cursor = self._create_cursor()
116
117             sql = 'SELECT "product_id", "name", "description", "price",
"stock", "category_id", "create_time", "update_time" FROM "商品基本信息表"
WHERE "product_id" = ?'
118             cursor.execute(sql, [product_id])
119
120             result = cursor.fetchone()
121             return result
122
123         except Exception as e:
124             print(f"❌ 查询失败: {e}")
125             return None
126         finally:
127             if cursor:
128                 cursor.close()
129
130     def create_product(self, name: str, description: str, price: float,
stock: int, category_id: Optional[int] = None) -> bool:
131         """
132         插入新商品
133         """
134         cursor = None
135         try:
136             cursor = self._create_cursor()
137
138             sql = '''INSERT INTO "商品基本信息表"
139                 ("name", "description", "price", "stock",
"category_id")
140                 VALUES (?, ?, ?, ?, ?)'''
141
142             cursor.execute(sql, [name, description, price, stock,
category_id])
143             self.conn.commit()
144
145             print(f"✓ 插入成功! 商品名称: {name}")
146             return True
147
148         except Exception as e:
149             print(f"❌ 插入失败: {e}")
150             self.conn.rollback()
151             return False
152         finally:
153             if cursor:

```

```

154         cursor.close()
155
156     def update_product(self, product_id: int, price: Optional[float] = None,
157 stock: Optional[int] = None) -> bool:
158         """
159         更新商品信息
160         """
161         cursor = None
162         try:
163             cursor = self._create_cursor()
164
165             # 动态构建 SQL
166             updates = []
167             params = []
168
169             if price is not None:
170                 updates.append('"price" = ?')
171                 params.append(price)
172
173             if stock is not None:
174                 updates.append('"stock" = ?')
175                 params.append(stock)
176
177             if not updates:
178                 print("⚠️ 没有需要更新的字段")
179                 return False
180
181             # 添加 update_time 更新
182             updates.append('"update_time" = NOW()')
183
184             sql = f'UPDATE "商品基本信息表" SET {", ".join(updates)} WHERE
185 "product_id" = ?'
186             params.append(product_id)
187
188             cursor.execute(sql, params)
189             self.conn.commit()
190
191             if cursor.rowcount > 0:
192                 print(f"✓ 更新成功! 受影响行数: {cursor.rowcount}")
193                 return True
194             else:
195                 print("⚠️ 未找到该商品")
196                 return False
197
198         except Exception as e:
199             print(f"✗ 更新失败: {e}")
200             self.conn.rollback()
201             return False
202         finally:
203             if cursor:
204                 cursor.close()
205
206     def delete_product(self, product_id: int) -> bool:
207         """
208         删除商品

```

```

207         """
208         cursor = None
209         try:
210             cursor = self._create_cursor()
211
212             sql = 'DELETE FROM "商品基本信息表" WHERE "product_id" = ?'
213             cursor.execute(sql, [product_id])
214             self.conn.commit()
215
216             if cursor.rowcount > 0:
217                 print(f"✓ 删除成功! 受影响行数: {cursor.rowcount}")
218                 return True
219             else:
220                 print(f"⚠ 未找到该商品")
221                 return False
222
223         except Exception as e:
224             print(f"✗ 删除失败: {e}")
225             self.conn.rollback()
226             return False
227         finally:
228             if cursor:
229                 cursor.close()
230
231
232     class MainMenu:
233         """用户交互主程序"""
234
235         def __init__(self, product_crud: ProductCRUD):
236             """初始化主菜单"""
237             self.product_crud = product_crud
238
239         def print_menu(self):
240             """打印操作菜单"""
241             print("\n" + "=" * 50)
242             print("          GaussDB 商品管理系统 v1.0")
243             print("=" * 50)
244             print("1. 查询所有商品")
245             print("2. 根据 ID 查询商品")
246             print("3. 插入新商品")
247             print("4. 更新商品信息")
248             print("5. 删除商品")
249             print("0. 退出程序")
250             print("=" * 50)
251
252         def print_products_table(self, products: List[Tuple]):
253             """以表格形式打印商品列表"""
254             if not products:
255                 print(f"\n📦 当前无商品记录\n")
256                 return
257
258             print("\n【查询结果】")
259             print("-" * 120)
260             print(f"{'ID':<6} {'商品名称':<20} {'描述':<30} {'价格':<10} {'库存':<8} {'分类ID':<8} {'创建时间':<20}")

```

```

261         print("-" * 120)
262
263         for product in products:
264             product_id, name, desc, price, stock, cat_id, create_time, _ =
product
265
266             # 所有字段都需要转换为 Python 原生类型避免 Java 对象格式化错误
267             pid_str = str(product_id) if product_id else ""
268             name_str = (str(name)[:19]) if name and len(str(name)) > 20 else
(str(name) if name else "")
269             desc_str = (str(desc)[:27] + "...") if desc and len(str(desc)) >
30 else (str(desc) if desc else "")
270             cat_str = str(cat_id) if cat_id else "N/A"
271             create_str = str(create_time)[:19] if create_time else ""
272
273             # 转换 price 和 stock 为 Python 原生类型
274             price_val = float(str(price)) if price else 0.0
275             stock_val = int(str(stock)) if stock else 0
276
277             print(f"{pid_str:<6} {name_str:<20} {desc_str:<30} {price_val:
<10.2f} {stock_val:<8} {cat_str:<8} {create_str:<20}")
278
279         print("-" * 120)
280         print(f"共 {len(products)} 条记录\n")
281
282     def handle_query_all(self):
283         """处理查询所有商品"""
284         print("\n【查询所有商品】")
285         products = self.product_crud.query_all_products()
286         self.print_products_table(products)
287
288     def handle_query_by_id(self):
289         """处理根据 ID 查询"""
290         print("\n【根据 ID 查询商品】")
291         try:
292             product_id = int(input("请输入商品 ID: "))
293             product = self.product_crud.query_product_by_id(product_id)
294
295             if product:
296                 self.print_products_table([product])
297             else:
298                 print("⚠️ 未找到该商品\n")
299
300         except ValueError:
301             print("❌ 输入无效, 请输入数字\n")
302
303     def handle_create(self):
304         """处理插入新商品"""
305         print("\n【插入新商品】")
306         try:
307             name = input("商品名称: ").strip()
308             description = input("商品描述: ").strip()
309             price = float(input("价格: "))
310             stock = int(input("库存: "))
311             category_id_str = input("分类 ID (留空表示 NULL): ").strip()

```

```

312         category_id = int(category_id_str) if category_id_str else None
313
314     if not name:
315         print("❌ 商品名称不能为空\n")
316         return
317
318     self.product_crud.create_product(name, description, price,
319 stock, category_id)
320
321     except ValueError:
322         print("❌ 输入格式错误\n")
323
324 def handle_update(self):
325     """处理更新商品"""
326     print("\n【更新商品信息】")
327     try:
328         product_id = int(input("请输入要更新的商品 ID: "))
329
330         price_str = input("新价格 (留空不改): ").strip()
331         stock_str = input("新库存 (留空不改): ").strip()
332
333         price = float(price_str) if price_str else None
334         stock = int(stock_str) if stock_str else None
335
336         self.product_crud.update_product(product_id, price, stock)
337
338     except ValueError:
339         print("❌ 输入格式错误\n")
340
341 def handle_delete(self):
342     """处理删除商品"""
343     print("\n【删除商品】")
344     try:
345         product_id = int(input("请输入要删除的商品 ID: "))
346
347         # 先查询确认
348         product = self.product_crud.query_product_by_id(product_id)
349         if not product:
350             print("⚠️ 未找到该商品\n")
351             return
352
353         print(f"\n即将删除商品: {product[1]}")
354         confirm = input("确认删除? (y/n): ").strip().lower()
355
356         if confirm == 'y':
357             self.product_crud.delete_product(product_id)
358         else:
359             print("✓ 已取消删除\n")
360
361     except ValueError:
362         print("❌ 输入格式错误\n")
363
364 def run(self):
365     """主循环"""

```



```

366         while True:
367             self.print_menu()
368
369             try:
370                 choice = input("请输入操作编号: ").strip()
371
372                 if choice == '1':
373                     self.handle_query_all()
374                 elif choice == '2':
375                     self.handle_query_by_id()
376                 elif choice == '3':
377                     self.handle_create()
378                 elif choice == '4':
379                     self.handle_update()
380                 elif choice == '5':
381                     self.handle_delete()
382                 elif choice == '0':
383                     print("\n【程序退出】感谢使用!")
384                     break
385                 else:
386                     print("\n❌ 无效的选项, 请重新输入\n")
387
388             except KeyboardInterrupt:
389                 print("\n\n【程序中断】用户取消操作")
390                 break
391             except Exception as e:
392                 print(f"\n❌ 发生错误: {e}\n")
393
394
395 def main():
396     """
397     主函数: 完整的数据库访问流程
398     """
399     # === 加载环境变量 ===
400     load_dotenv()
401
402     # === 从环境变量读取配置信息 ===
403     jar_path = os.getenv('GAUSSDB_JAR_PATH')
404     host = os.getenv('GAUSSDB_HOST')
405     port = os.getenv('GAUSSDB_PORT')
406     database = os.getenv('GAUSSDB_DATABASE')
407     user = os.getenv('GAUSSDB_USER')
408     password = os.getenv('GAUSSDB_PASSWORD')
409
410     # 构建 JDBC URL
411     jdbc_url = f"jdbc:gaussdb://{host}:{port}/{database}"
412
413     db_conn = None
414
415     try:
416         # Step 1 & 2: 初始化并连接数据库
417         db_conn = DBConnection(jar_path, jdbc_url, user, password)
418         connection = db_conn.connect()
419
420         print("\n" + "=" * 50)

```

```
421         print("【Step 3】开始执行数据库操作...")
422         print("=" * 50)
423
424         # Step 3: 创建 CRUD 操作对象并启动主菜单
425         product_crud = ProductCRUD(connection)
426         main_menu = MainMenu(product_crud)
427         main_menu.run()
428
429     except Exception as e:
430         print(f"\n❌ 程序异常: {e}")
431
432     finally:
433         # Step 4: 清理资源
434         print("\n" + "=" * 50)
435         print("【Step 4】释放资源...")
436         print("=" * 50)
437
438         if db_conn:
439             db_conn.close()
440
441
442 if __name__ == "__main__":
443     main()
```

7 附录B：演示程序源代码

```
1 import jaydebeapi
2 import jpyype
3 import os
4 from datetime import datetime
5 from dotenv import load_dotenv
6
7
8 def demo_crud_operations():
9     """演示数据库 CRUD 操作的完整流程"""
10    # === 配置信息 ===
11    # 从环境变量读取配置
12    load_dotenv()
13
14    # JAR 路径
15    jar_path = os.getenv(
16        'GAUSSDB_JAR_PATH',
17        os.path.abspath(os.path.join(os.path.dirname(__file__), '..',
18        'driver', 'gaussdbjdbc.jar'))
19    )
20
21    driver_class = "com.huawei.gaussdb.jdbc.Driver"
22
23    # JDBC URL
24    jdbc_url = os.getenv('GAUSSDB_JDBC_URL')
25    if not jdbc_url:
26        host = os.getenv('GAUSSDB_HOST')
27        port = os.getenv('GAUSSDB_PORT')
28        database = os.getenv('GAUSSDB_DATABASE')
29        if host and port and database:
30            jdbc_url = f"jdbc:gaussdb://{host}:{port}/{database}"
31        else:
32            raise RuntimeError("数据库连接信息不完整：请设置 GAUSSDB_JDBC_URL 或
33            GAUSSDB_HOST/GAUSSDB_PORT/GAUSSDB_DATABASE")
34
35    user = os.getenv('GAUSSDB_USER')
36    password = os.getenv('GAUSSDB_PASSWORD')
37    if not user or not password:
38        raise RuntimeError("缺少数据库用户名或密码，请设置 GAUSSDB_USER 和
39        GAUSSDB_PASSWORD")
40
41    conn = None
42
43    try:
44        # === Step 1: 初始化 JDBC 环境 ===
45        print("\n" + "=" * 80)
46        print("【Step 1】初始化 JDBC 环境")
47        print("=" * 80)
48
49        if not jpyype.isJVMStarted():
50            jpyype.startJVM(
51                jpyype.getDefaultJVMPath(),
```

```

49         "-ea",
50         f"-Djava.class.path={jar_path}",
51         "-Djava.util.logging.config.file=logging.properties"
52     )
53     print("✓ JVM 启动成功")
54
55     # === Step 2: 建立数据库连接 ===
56     print("\n" + "=" * 80)
57     print("【Step 2】建立数据库连接")
58     print("=" * 80)
59
60     conn = jaydebeapi.connect(
61         driver_class,
62         jdbc_url,
63         [user, password],
64         jar_path
65     )
66     print("✓ 数据库连接成功")
67
68     # 关闭自动提交模式，支持手动事务管理
69     conn.jconn.setAutoCommit(False)
70     print("✓ 已切换为手动事务模式")
71
72     # === Step 3: 执行 CRUD 操作 ===
73     print("\n" + "=" * 80)
74     print("【Step 3】执行数据库 CRUD 操作")
75     print("=" * 80)
76
77     # 3.1 第一次查询：打印所有记录
78     print("\n[操作 1] 查询所有商品（初始状态）")
79     print("-" * 80)
80     query_all(conn)
81
82     # 3.2 插入新商品
83     print("\n[操作 2] 插入测试商品")
84     print("-" * 80)
85     test_product_name = f"测试商品_{datetime.now().strftime('%H%M%S')}"
86     insert_product(conn, test_product_name, "这是一个测试商品", 999.99,
100, 1)
87
88     # 3.3 第二次查询：验证插入
89     print("\n[操作 3] 再次查询（验证插入成功）")
90     print("-" * 80)
91     query_all(conn)
92
93     # 获取刚插入的商品 ID
94     product_id = get_product_id_by_name(conn, test_product_name)
95
96     if product_id:
97         # 3.4 修改商品
98         print(f"\n[操作 4] 修改商品 ID={product_id}（价格改为 1999.99，库存改
100 为 50）")
99         print("-" * 80)
100         update_product(conn, product_id, 1999.99, 50)
101

```

```

102         # 3.5 第三次查询：验证修改
103         print("\n[操作 5] 再次查询（验证修改成功）")
104         print("-" * 80)
105         query_all(conn)
106
107         # 3.6 删除商品
108         print(f"\n[操作 6] 删除商品 ID={product_id}")
109         print("-" * 80)
110         delete_product(conn, product_id)
111
112         # 3.7 第四次查询：验证删除
113         print("\n[操作 7] 最终查询（验证删除成功）")
114         print("-" * 80)
115         query_all(conn)
116
117     except Exception as e:
118         print(f"\n❌ 发生错误：{e}")
119         import traceback
120         traceback.print_exc()
121
122     finally:
123         # === Step 4: 释放资源 ===
124         print("\n" + "=" * 80)
125         print("【Step 4】释放资源")
126         print("=" * 80)
127
128         if conn:
129             conn.close()
130             print("✓ 数据库连接已关闭")
131
132         if jpyype.isJVMStarted():
133             jpyype.shutdownJVM()
134             print("✓ JVM 已关闭")
135
136         print("\n" + "=" * 80)
137         print("【演示完成】")
138         print("=" * 80)
139
140
141     def query_all(conn):
142         """查询所有商品"""
143         cursor = None
144         try:
145             cursor = conn.cursor()
146             sql = 'SELECT "product_id", "name", "description", "price", "stock",
147 "category_id" FROM "商品基本信息表" ORDER BY "product_id"'
148             cursor.execute(sql)
149
150             results = cursor.fetchall()
151
152             if not results:
153                 print("📦 当前无商品记录")
154                 return

```

```

155         print(f"\n{'ID':<6} {'商品名称':<25} {'描述':<35} {'价格':<10} {'库存':<8} {'分类ID':<8}")
156         print("-" * 100)
157
158         for row in results:
159             pid = row[0]
160             name = str(row[1])[:24]
161             desc = str(row[2])[:34] if row[2] else ""
162             price = float(str(row[3]))
163             stock = int(str(row[4]))
164             cat = str(row[5]) if row[5] else "N/A"
165
166             print(f"{pid:<6} {name:<25} {desc:<35} {price:<10.2f} {stock:<8} {cat:<8}")
167
168         print("-" * 100)
169         print(f"共 {len(results)} 条记录\n")
170
171     finally:
172         if cursor:
173             cursor.close()
174
175
176 def insert_product(conn, name, desc, price, stock, category_id):
177     """插入商品"""
178     cursor = None
179     try:
180         cursor = conn.cursor()
181         sql = '''INSERT INTO "商品基本信息表"
182             ("name", "description", "price", "stock", "category_id")
183             VALUES (?, ?, ?, ?, ?)'''
184
185         cursor.execute(sql, [name, desc, price, stock, category_id])
186         conn.commit()
187
188         print(f"✓ 插入成功! 商品名称: {name}, 价格: {price}, 库存: {stock}")
189
190     except Exception as e:
191         print(f"✗ 插入失败: {e}")
192         conn.rollback()
193     finally:
194         if cursor:
195             cursor.close()
196
197
198 def get_product_id_by_name(conn, name):
199     """根据名称获取商品 ID"""
200     cursor = None
201     try:
202         cursor = conn.cursor()
203         sql = 'SELECT "product_id" FROM "商品基本信息表" WHERE "name" = ?'
204         cursor.execute(sql, [name])
205
206         result = cursor.fetchone()
207         return result[0] if result else None

```

```
208
209     finally:
210         if cursor:
211             cursor.close()
212
213
214 def update_product(conn, product_id, price, stock):
215     """更新商品"""
216     cursor = None
217     try:
218         cursor = conn.cursor()
219         sql = 'UPDATE "商品基本信息表" SET "price" = ?, "stock" = ?,
220 "update_time" = NOW() WHERE "product_id" = ?'
221
222         cursor.execute(sql, [price, stock, product_id])
223         conn.commit()
224
225         print(f"✓ 更新成功! 商品 ID: {product_id}, 新价格: {price}, 新库存:
226 {stock}")
227
228     except Exception as e:
229         print(f"✗ 更新失败: {e}")
230         conn.rollback()
231     finally:
232         if cursor:
233             cursor.close()
234
235
236 def delete_product(conn, product_id):
237     """删除商品"""
238     cursor = None
239     try:
240         cursor = conn.cursor()
241         sql = 'DELETE FROM "商品基本信息表" WHERE "product_id" = ?'
242
243         cursor.execute(sql, [product_id])
244         conn.commit()
245
246         if cursor.rowcount > 0:
247             print(f"✓ 删除成功! 商品 ID: {product_id}")
248         else:
249             print(f"⚠ 未找到商品 ID: {product_id}")
250
251     except Exception as e:
252         print(f"✗ 删除失败: {e}")
253         conn.rollback()
254     finally:
255         if cursor:
256             cursor.close()
257
258
259 if __name__ == "__main__":
260     demo_crud_operations()
```

